Data Wrangling

- Data manipulation & Cleaning
- Calculate Summary Statistics
- Visualization
- Input for Modeling

* dplyr : Tools for data wrangling

- part of the tidyverse :
  - filter() → subset of rows
  - select() : take a subset of the columns
  - group by ()
  - mutate : add or modify columns
  - arrange() : sort
  - summary ()

ie

Do not change original dataset

```
filter ( starwars, species == "Droid")
```

```
starwars |> ← pipe
    filter ( species == " Droid")
```

same function

```
    summarize ( mean_height = mean ( height))
```

↳ calculate summary statistics

```
mean ( height, na.rm = T )
```

```
starwars |>
    group_by ( species) |>
    summarize ( mean_height = mean( height , na.rm=T))
```

for each groups

```
starwers |>
    mutate ( body_ratio = mass / height)
```

create new column

```
    group_by ( species) |>
    summarize ( mean_ratio = mean ( body_ratio, na.rm = T), sd_ratio = sd ( body_ratio, na.rm = T),
            N= n() )
```

↑ counts rows

```
mutate ( gm = case_when (
        yearID <= 2003 ~ "Philips",
        yearID == 2004 ~ "Duquette"
        yearID <= 2010 ~ "Minaya" ))
```

* Data Wrangling Fundamentals

```
group_by(gm) |>
summarize(wpct = sum(w)/sum(w+l))
arrange(desc(wpct))
```

* if arrange(wpct) → get lowest to highest

Narrow vs. Wide.

```
df |>
  pivot_longer(
      cols = bp1:bp2,
      names_to = "measurement",
      value_to = "value"
  )
```

id	bp1	bp2
A	150	158

↓

id	measurement	value
A	bp1	150
A	bp2	158

cols = -country ← everything except country

```
|>
drop_na()
```

bp_1 → bp 1

↳ separate names

```
names_to = c("measurement", "stage"),
names_sep = "_",
values_to = "value"
```

linear model: lm(x ~ y, data = ___)

".value" here keeps the first part as the column

values_to = " " → create name for values column

- pivot_wider(id_cols = ..., ← columns to keep
  names_from = ..., ← new column names
  values_from = ...) ← values to fill in

Sept 8th, Joins

```
patients |>
    left_join (doctors, join_by (provider))
```

Different Column Names:

provider == name

from modified dataset

Across function:

```
diamonds |>
    summarize ( across (c( carat, depth, price), mean))
```

Columns to summarize

function to apply to each column

```
list (mean, sd)  ⟶  list ( "mean"=mean , "sd" =sd)
```

specify a list of all functions to apply

```
    .names = "{.col} _ {.fn} ")

Summarize ( across ( where ( is. numeric)
```

↳ check

```
summarize ( across ( where ( is. factor ) .
```

categorical variables

```
        list ( "num _ categories" = n_distinct ) ))
```

No. of distinct values

- Ignoring NAs

```
list ("median" = median (          ))
```

Write a function:

```
* median_no_na <- function (x) {
    median (x, na.rm =T)
  }

    list ("median" = median_no_na)
```

* Functions

```
z_score <- function (x) {
    (x- mean (x, na.rm = TRUE))/ sd (x, na.rm= TRUE)
}
```

"mutate function"

```
x -> z_score(x)
```

```
find_outliers <- function (x) {
```

in function:

use {{ group_var }} to refer to variables

lm function does not support tidy evaluation

```
grade_tables <- map ( grade_files , read_CSV)
```

list — iteration object — function

```
map ( function (df) slr_slope (df, midterm1, midterm2))
```

```
map (x, function)
```

```
function (x)      quantile ( x, 0.5 )
```

```
nsim = 1000
hats <- rep (NA, 3)
```

- \d any digit
- + one or more occurrences
- ^ at the beginning
- $ at the end
- \b word boundary
- | or

* str_remove_all(____, "\\D")
  - \d → any digit
  - \D → any non-digit
  - \w → word characters
  - \s white space character
  - \D, \W, \S works as well

str_subset(filenames, "^research")

(filenames, "^research.+\\.csv$")

special character that matches any character

str_view(strings, "r{3}")

↳ 3 "r"s

any character

(strings, "(.)\\1")

capture group

Back reference

↳ one occurrence of any character

chart

berry

papa, banana, momento

↳ str_view(strings, "(..)\\1")

<papa>

b<anan>a ...

(..)\\1

a group with two

* extract things in between $s

"\\$.+\\$" ↗ largest match

↳ "\\$.+?\\$

.+?\\$

don't be greedy

→ No dollar sign in the middle of it

"\\$[^\\$]+\\$"

[^] → everything except the dollar sign

* Web Scraping

read_html

in html: src = source file

<html> : start of the HTML page

<head> : header information

<body> : everything that is on the page

<h1> to <h6> : headings

<p> : paragraphs

<b> : bold

<table> : table

html_elements("h3[id^='Episode']")|>

html_text2()

hyper link → href

div > a → find child of div

* Intro to SQL

SQL

data too big to store locally → access remotely

query → string

dbGetQuery ← to run query remotely & give us the results

SELECT : select, mutate, summarize

AS to assign new name

SUM

SELECT

FROM

LIMIT

FROM Flights → table that we are using from the select

JOIN → join columns

WHERE → filtering

in SQL, No "==" → assignment → "AS"

LIMIT 0, 4;

↳ first four rows shown

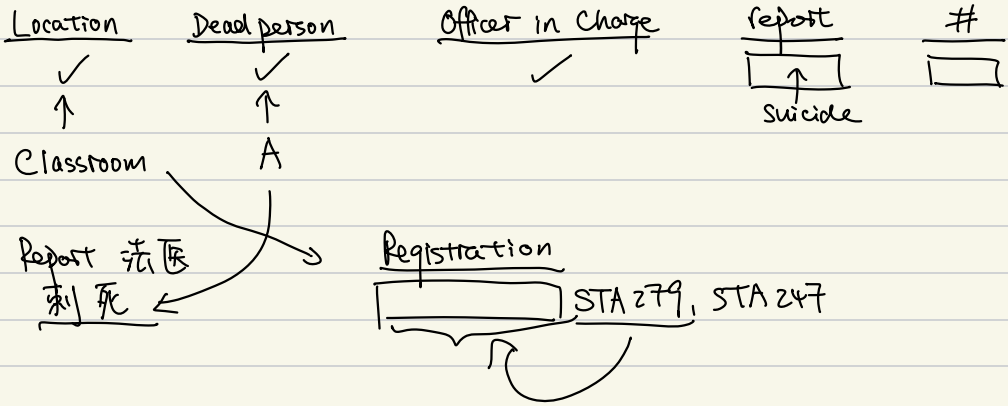
Required Clauses:

SELECT, FROM

* Homework 8

identification

ID ✓
Name
Age
Relationship
income
hair color



Final Review : Python

choose columns: titanic[['Pclass', 'Survived']]

or use titanic.filter([ ])

choose: titanic[titanic['Pclass'] == 1]

sort_values(by = " ", ascending =)

pd.merge(, df, df, left_on

Do Summary Statistics:

titanic.agg({'survived': 'mean'})

['mean', 'std']

(titanic.groupby(by = ['Pclass', 'Sex']).agg(survival_rate = ('survived', 'mean')))

create new variable

np.arange(M) → 1.... M

np.random.choice(hats, M, replace = False)

np.zeros(nsim)

np.mean(results)

pivot longer cols names_to

names_sep

values_to

-pivot_longer(

cols = bp1 : bp2,

names_to = "measurement",

values_to = "value"

) values_drop_na = T

pivot_longer(cols= -id,

names_to = c("group", "obs"),

names_sep = "_",

values_to = "value")

pivot_longer(cols= -id,

names_to = c(".value", "stage"),

names_sep = "_")

pivot_wider(id_cols = ...,

names_from = ...,

values_from = ...)

* Join

left_join(doctors, Join_by(provider))

Join_by(provider == name)

inner_Join(insurance, Joins_by(insurance == company))

* Across

summarize (across(c c carat, depth, price), mean))

list(mean, sd), .names = "{.fn} _ {.col}"))

name in the list

across (where (is.numeric). ____))

is.factor → categorical data

summarize (across (starts_with ("a"), mean))

list("...") → col _ ...

in function, refer to variable in pipeline: {{ group_var }} (lm not support)

randomized_hats ← sample(hats, n, replace = False)

c ( ) → c[1]

x← list ( ) → x[[1]]

\d any digit \D → any non digit

^ at the beginning

$ at the end

\b word boundary

\w → word character

\s white space

SQL: LEFT JOIN ON

WHERE LIKE

HAVING → new column

ORDER BY

by =

df. groupby("). agg(____ = (" ", " "))